Defense 4, in full detail: what each mode is, how the mechanism works, and how it is built

This is the detailed tutorial. It explains, from the ground up, what D1, D2, D3, and D4 actually do, the mechanism that makes them possible on a switch, and how the whole thing is implemented in P4 and in the control and measurement code. It is long on purpose. Read it top to bottom and you should be able to explain Defense 4 to someone else.

Part I — The problem and the idea

1. The fingerprint we are hiding

A DNP3 master polls an outstation (here, a physical SEL-751 protective relay) with a READ. The relay answers in two separate pieces on the wire:

1. a **pure TCP acknowledgment** (an empty TCP segment with the ACK flag), sent by the relay's TCP stack soon after it receives the READ, and
2. a little later, the **DNP3 RESPONSE** (a TCP segment carrying the DNP3 application data).

The gap between those two, measured at the master, is the **cross-layer response time, the CLRT**. It is the time from the pure TCP acknowledgment to the first byte of the matching DNP3 response. Every device model has its own habitual CLRT, set by its firmware and its processing. A passive observer who simply watches the traffic, decoding nothing, can measure the CLRT over many polls and use its shape as a fingerprint: "this timing belongs to an SEL-751." That is reconnaissance. We measured the SEL-751's native CLRT: its middle 90 percent spans about 1.8 ms to 7.6 ms, a wide, telltale spread.

2. The constraints that make this hard

We cannot fix this at the device or in the protocol:

- The relay is vendor firmware. We cannot reprogram it.
- DNP3 is a deployed protocol with a plaintext, CRC-protected frame. We cannot change it, and we cannot inject fake content or edit bytes, because the checksum and the plaintext make tampering detectable.
- The defense must be transparent: same two endpoints, same request-and-response exchange, same bytes, same correctness. The master must still get its real answer.

So the defense has to live in the **network**, on a switch between the master and the relay, and it may only change **when** things become visible, never **what** is visible.

3. The one hard fact about a switch data plane

A programmable switch is not a general computer. Its data plane processes each packet in a few nanoseconds as the packet flies through a fixed pipeline. **It cannot take a packet, put it to sleep, and wake it up 10 milliseconds later at a time chosen in software.** There is no "sleep and recall." Once a packet is enqueued for a port, the traffic manager decides when it leaves based on queue priority and scheduling, not on a software timer that can reach back in and grab a specific packet.

Every design decision in Defense 4 follows from working *with* this fact instead of against it.

Part II — The mechanism

4. Hold the real packet, race it against a token

The trick is to make the switch's own scheduler do the waiting, using two ingredients:

1. **Queue residency.** When the relay's acknowledgment (or response) arrives, we do not forward it. We send it to a **low-priority hold queue** where it sits, still the original unmodified packet.
2. **Blocker tokens.** At the same time, the switch generates small internal packets (the "blocker tokens," carrying a private EtherType 0x88C1) and puts them in a **high-priority blocker queue**. These tokens recirculate through the pipeline over and over. As long as a blocker token outranks the held packet, the scheduler keeps serving the token and the held packet stays put.

Each time a blocker token comes around, the data plane checks the clock against the deadline. While the deadline has not passed, the token is re-sent to keep blocking. When the deadline passes, the token stops blocking (it is retired), the held packet is now the highest-priority thing left in its direction, and the traffic manager releases it. The release time is therefore set by the deadline, even though no software ever reached in and grabbed the packet.

The blocker tokens never leave toward the master; they only ever recirculate internally. The original packet leaves unchanged.

5. The four queues and their strict priority

There are four queues, in a strict priority order (higher number wins):

- **Q_ACK_BLOCK, qid 7** — holds the acknowledgment's blocker tokens (highest).
- **Q_ACK_HOLD, qid 6** — holds the real acknowledgment.

- **Q_RESP_BLOCK, qid 5** — holds the response's blocker tokens.
- **Q_RESP_HOLD, qid 4** — holds the real response (lowest).

Strict priority means the scheduler always serves a non-empty higher queue before a lower one. Two consequences matter:

- While an acknowledgment blocker (qid 7) is recirculating, the real acknowledgment (qid 6) cannot leave. When the acknowledgment deadline passes and the blocker retires, qid 6 is served and the acknowledgment is released.
- Because the acknowledgment queues (7, 6) outrank the response queues (5, 4), the acknowledgment is **never** released after the response. Ordering is guaranteed by the queue structure, not by luck.

6. The transaction lifecycle and the generation

The switch must know which acknowledgment and response belong to which READ, and it must not confuse one poll with the next. It uses the DNP3 **application-control octet**, which the master advances C0, C1, … CF and then rolls back to C0. We call this the **generation**. One stateful register, `reg_tag`, remembers the generation of the currently active transaction.

The lifecycle of one transaction:

1. **Arm.** A READ is classified; `reg_tag` records its generation. The switch also triggers the pktgen to create the blocker tokens for this transaction.
2. **Hold the acknowledgment.** The relay's acknowledgment arrives and goes to Q_ACK_HOLD; its blocker recirculates in Q_ACK_BLOCK, checking the acknowledgment deadline T_A.
3. **Release the acknowledgment.** At T_A the acknowledgment blocker retires and the acknowledgment is released.
4. **Hold the response.** The relay's response goes to Q_RESP_HOLD; its blocker recirculates in Q_RESP_BLOCK, checking the response deadline T_RESP.
5. **Release the response.** At T_RESP the response blocker retires and the response is released.
6. **Retire and re-arm.** The transaction retires; the next READ (next generation) arms a fresh one.

7. The timing math

Let `t_A` be the moment the acknowledgment arrives. Two configured delays set the deadlines:

- `D_A` (acknowledgment delay) gives the acknowledgment deadline **T_A = t_A + D_A**.
- `D_R` (response delay) gives the response deadline **T_RESP = t_A + D_A + D_R**.

So `D_R` is the intended CLRT: the interval from the released acknowledgment to the released response. Delays are encoded as a "tick" word (one tick is about one nanosecond) shifted so the low byte is zero; for example 10 ms is the word 9,999,872. The control plane sets these; the data plane only reads them.

A response can arrive in one of four places, and all four are handled safely:

- **before T_A** (the relay was fast): held in Q_RESP_HOLD, released at T_RESP. Counter RESP_HOLD_EARLY.
- **between T_A and T_RESP** (after the acknowledgment was already released): still held to T_RESP. Counter RESP_HOLD_LATE. This is the case the lifecycle bug used to get wrong.
- **after T_RESP** (the relay was slow): released as soon as it arrives, a **late safe release**. This is not deadline normalization; it is honest slack, and we report it as a tail.
- **after the fail-open horizon** (the reservoir of blocker tokens drains first): released early by the **fail-open** path so nothing is ever stranded. Counter RELEASE_FAILOPEN.

Part III — The five modes, each explained

OFF is the baseline: forward everything unchanged, so the observer sees the native CLRT. The four defenses are all the same machinery with different gates turned on.

8. D1 — event mode

Idea: release the acknowledgment only when the matching response actually shows up.

Mechanism: the acknowledgment is held in Q_ACK_HOLD. Instead of a fixed acknowledgment deadline, the release is triggered by the **event** of the matching response arriving (or a watchdog). So the acknowledgment and the response come out close together, tracking the response's real arrival.

What it achieves and its limit: D1 removes the *independent* acknowledgment-timing signal by tying the acknowledgment to the response. But because it tracks the response's real arrival, it **shifts** the timing without **tightening** it. In our data D1 moved the CLRT up to a median of about 11 ms but kept a 5.17 ms spread, only 0.24 bits less entropy than OFF. D1 is the weakest normalizer, and we say so. It is useful when you want event coupling without committing to a fixed deadline.

9. D2 — response-deadline mode

Idea: let the acknowledgment go immediately, then hold the response to a fixed deadline after it.

Configuration: `D_A = 0`, so the acknowledgment deadline T_A equals t_A and the acknowledgment is released at once. `D_R = 10 ms`, so the response is held until T_RESP = t_A + 10 ms.

Mechanism: the acknowledgment is released immediately (no acknowledgment hold). The response goes to Q_RESP_HOLD and its blocker recirculates in Q_RESP_BLOCK until T_RESP, then it is released. Because the acknowledgment left at t_A and the response leaves at t_A + 10 ms, the CLRT the observer measures is a fixed 10 ms, whatever the relay's real processing time was.

Why it is the pure normalizer: every response, fast or slow (up to the deadline), leaves at the same 10 ms mark. In our data D2's CLRT p5-p95 was 9.95 to 10.08 ms, a 0.12 ms band, a 45.6x reduction from OFF's 5.69 ms spread, with 0 bypass across 240 transactions. Note that D2 depends entirely on the response obligation surviving the acknowledgment release, because with D_A = 0 the acknowledgment is already gone when every response arrives. That is exactly why the lifecycle bug hit D2 hardest (it bypassed all 240 on the broken binary).

10. D3 — acknowledgment-deadline mode

Idea: hold the acknowledgment to a fixed deadline, then let the response go as soon as it is ready.

Configuration: `D_A = 4 ms`, `D_R = 0`, so T_RESP = T_A. This is the original Defense 3 policy.

Mechanism: the acknowledgment is held in Q_ACK_HOLD until T_A = t_A + 4 ms. The response deadline equals the acknowledgment deadline, so when the acknowledgment is released the response (which for this fast relay has usually already arrived) is released right behind it, or a response that arrives after the acknowledgment deadline is simply forwarded. The acknowledgment and response therefore come out together.

What it achieves: the CLRT collapses toward zero, because the acknowledgment and response leave at almost the same instant. In our data D3's CLRT median was 0.03 ms; its acknowledgment and response timestamps coincide at capture resolution (this is *why* the scorer had to be made mode-aware: for D3 that coincidence is the design, not an error). D3 removes the CLRT by squeezing it to nothing, rather than by pinning it to a fixed positive value. A response that arrives after the acknowledgment deadline is forwarded rather than held; that is a legitimate D3 "bypass" (D_R = 0), not the lifecycle defect.

11. D4 — dual-deadline mode

Idea: hold both. Release the acknowledgment at a fixed acknowledgment deadline, and independently hold the response to a fixed response deadline.

Configuration: `D_A = 4 ms`, `D_R = 10 ms`, so T_A = t_A + 4 ms and T_RESP = t_A + 14 ms, and the observed CLRT (released acknowledgment to released response) is D_R = 10 ms.

Mechanism: both gates are on. The acknowledgment is held in Q_ACK_HOLD to T_A; the response is held in Q_RESP_HOLD to T_RESP. Both are released on their own deadlines. D4 gives you a fixed acknowledgment timing *and* a fixed CLRT.

The hard part D4 exposes. With D_A = 4 ms, the acknowledgment is released 4 ms after it arrives, but the relay's real response often arrives *after* that release (in our data 27 to 33 of every 120 responses arrived after the acknowledgment was already gone). The switch must still be holding a live transaction for that response when it arrives. This is the "the response obligation survives the acknowledgment release" requirement, and it is exactly what the lifecycle bug broke. In our data D4's CLRT p5-p95 was 9.98 to 10.03 ms, a 0.05 ms band, a **118x** reduction from OFF, with 0 bypass across 240 transactions.

12. The bug that was fixed, in detail

The pre-fix program retired the transaction whenever the acknowledgment was released. That is correct for D1 and D3 (acknowledgment-only shaping: once the acknowledgment is dealt with, the transaction is done). It is wrong for D2 and D4, where a response is still owed after the acknowledgment release. On the broken binary a response that arrived after the acknowledgment release found a dead transaction and slipped through unshaped: D2 bypassed 240/240, D4 bypassed 80/240.

The fix has two halves, both small and resource-safe (no new register, stateful ALU, header field, or counter; the `reg_tag` register keeps its four actions):

1. **Mode-aware retirement.** The acknowledgment-release retirement now runs only for the acknowledgment-only modes D1 and D3. For the must-hold modes D2 and D4, the acknowledgment release keeps the transaction alive so the later response is still held. A read-only companion action on the acknowledgment register restores the early-versus-late response distinction without a write hazard, and the acknowledgment-release counter splits (ACK_RELEASE vs ACK_REL_RETIRE) so the evidence shows which path ran.
2. **A response blocker that does not vanish.** The response blocker (qid 5) now drains on the deadline only when a response is actually pending; otherwise it loops to the bounded budget instead of disappearing at T_RESP. A missing or late response no longer strands the reservoir.

13. Fail-open: never strand a packet

Holding is only safe if it always ends. Each transaction has a **budget** of blocker-token recirculations, which sets a **fail-open horizon**. If the reservoir of blocker tokens drains before the release condition is met (for example, a response that never comes), the held packet is released early by the fail-open path, and the next transaction re-arms cleanly. We proved this on silicon: with a small budget forcing a 1.37 ms horizon, all 30 responses were still delivered, all 30 via the fail-open path, every transaction re-armed, and no stale state was left behind.

Part IV — The implementation

14. The P4 program (`defense4/timing/p4/defense4_caseA.p4`, ~3000 lines)

- **Parser and direction.** The parser sorts each packet by origin: from the relay, from the master, from the recirculation loopback (blocker tokens coming around), or from the packet generator (freshly minted blocker tokens). Direction drives the forwarding port and the classification.
- **Policy tables.** `tbl_params` holds the run-time policy (mode, the deadline words D_A and D_A+D_R, the budget, the read length). `tbl_session` pins the one protected flow. The control plane writes these between drained transactions; the data plane only reads them.
- **The `reg_tag` transaction register.** One stateful register tracks the active generation. It has exactly four RegisterActions (arm-once, read-modify-write, read-or-mark, retire-if-unmarked). The design cannot add a fifth on this stateful ALU, which is why the fix had to reuse existing operands.
- **Queue selection.** Based on packet class (blocker versus original, acknowledgment versus response) and the deadline check, each packet is assigned qid 7, 6, 5, or 4. Blocker tokens recirculate on dp8.
- **Counters.** `ctr_fresh` records the fresh path (ARM_FRESH, RESP_HOLD_EARLY, RESP_HOLD_LATE, RESP_BYPASS, …); `ctr_deq` records the dequeued path (the release causes and the acknowledgment- release split). These are the counters the scorer reconciles against the wire.

15. The control plane (`defense4/timing/control/defense4_caseA_setup.py`)

One authority for the policy. Its operations: `initialize` (establish the fixed function once), `set-policy` (mode and delays, refuses while a transaction is active), `clear-evidence`, `verify-only` / `evidence-dump` / `snapshot` (read-only), `restore-only`. Delays are given in milliseconds and quantized to the tick encoding. No other code writes the policy table directly.

16. The measurement harness (`defense4/timing/control/deploy/`)

- **`score_campaign.py`** — the fail-closed, mode-aware scorer. It exits with an error on any hard anomaly (a must-hold bypass, an ordering inversion, a stale tag, counters that do not reconcile, a token on the wire, a queue or port drop, a missing or invalid capture, an absent counter) and passes only a fully valid block. A declared negative case must actually be exercised.
- **`campaign_driver.py`** — runs on the master, one sustained connection, 60 READs advancing C0..CF, full-Ethernet capture, one rich JSON row per poll.
- **`pair_bytes.py`** — matches the same frame at the relay-facing and master-facing capture points and compares the bytes, to prove the switch released exactly what it received.
- **`analyze_campaign.py`** — condition-aware statistics with full distributions and a session bootstrap.
- **`run_campaign.sh`** / **`manual_campaign.sh`** — the orchestrators (the manual one has no rollback trap, so a scorer flag cannot cost the deployment).
- **`make_manifest.sh`** and the **`fixtures/`** suite (78 tests) make the whole pipeline fail closed.

17. The controlled outstation (`defense4/timing/control/outstation/`)

`software_outstation.py` is a deterministic DNP3 software outstation for the negative-test lab. Its `plan()` function emits exactly the frames each of 21 controlled cases needs (missing acknowledgment, missing response, FIN/RST at three points, combined response, multi-segment response, SELECT, OPERATE, and so on). It is validated offline by 58 checks. SELECT and OPERATE only ever go to this software outstation, never to the physical relay.

Part V — What the experiments showed

18. The campaigns

Two campaigns on the corrected binary against the physical SEL-751, READ-only: Campaign A in fixed order, Campaign B in randomized order (recorded seed). Five modes, two sustained sessions per mode, 60 READs per session: 120 valid transactions per mode per campaign, 1200 in total. Every block passed the fail-closed scorer.

19. The numbers (n = 240 per mode, both campaigns)

mode	median CLRT	p5-p95 spread	spread reduction	entropy (bits)	effective states	bypass
OFF	2.92 ms	5.69 ms	1x	3.63	12.4	n/a (passthrough)
D1	11.14 ms	5.17 ms	1.1x	3.39	10.5	0
D2	10.02 ms	0.12 ms	45.6x	1.23	2.3	0
D3	0.03 ms	1.05 ms	5.4x	0.76	1.7	(D_R=0 forwards)
D4	10.00 ms	0.05 ms	118.2x	1.10	2.1	0

D2 and D4 collapse the observer's timing information from about 12 effective states to about 2. Targeted tests confirmed the response-survives-acknowledgment-release lifecycle, the C0..CF rollover on one connection, and the fail-open bounded release.

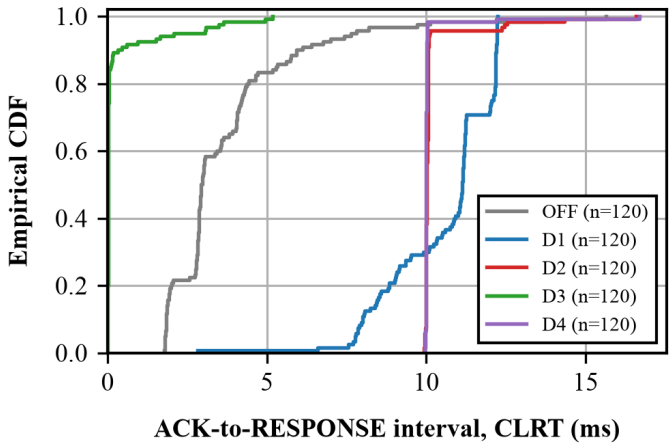

Figure 1: Empirical CDF of the CLRT. OFF (grey) is the wide native fingerprint; D2 (red) and D4 (purple) are near-vertical steps at 10 ms with a short late tail; D3 (green) collapses to ~0; D1 (blue) shifts to ~11 ms.

20. What is proven, and the boundary

Proven, corrected binary, fail-closed, reproduced: D2 and D4 normalize the CLRT to a fixed 10 ms (bulk, with an honest late tail), hold every response, and collapse the timing information for this device. Not done: the controlled negatives live through the switch (the outstation is built and offline-validated), the paired byte identity on a live dual capture, and cross-device classification

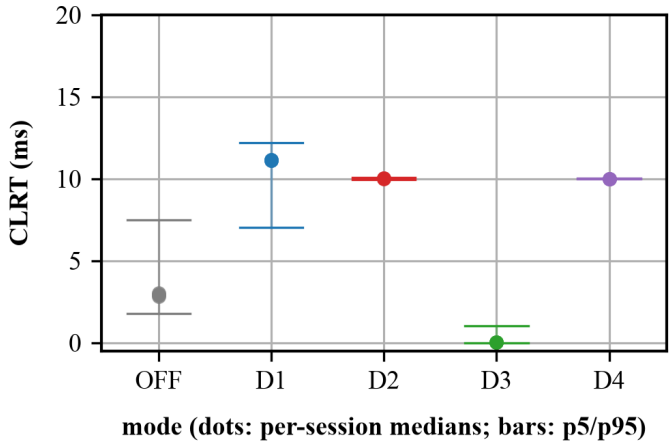

Figure 2: Per-session medians (dots) and pooled p5/p95 (bars) per mode. D2 and D4 sit tight at 10 ms and are stable across sessions; D1 shifts but stays wide; D3 collapses to ~0.

(needs a second comparable device). No final acceptance verdict; the paper Introduction stays quarantined.

Glossary

- **CLRT** — cross-layer response time: acknowledgment-to-response interval, master-facing.
- **Generation** — the DNP3 application-control octet C0..CF, used to match a transaction.
- **Blocker token** — an internal 0x88C1 packet that recirculates to hold a queue's place.
- **Queue residency** — the original packet waits in a switch queue, unmodified, until released.
- **T_A, T_RESP** — the acknowledgment and response deadlines (t_A + D_A, and + D_R).
- **Fail-open** — releasing a held packet early when the reservoir drains, so nothing is stranded.
- **reg_tag** — the stateful register that tracks the active transaction's generation.